AMERICAN INTERNATIONAL UNIVERSITY BANGLADESH (AIUB)

FACULTY OF SCIENCE & TECHNOLOGY

INTRODUCTION TO DATABASE

Fall 2023-2024
Section: M

TITLE

Doctor Appointment Management System

Supervised By
MD Sajid Bin Faisal

Submitted By:
Group no: 03

Name	ID
Shahriar Hossain	22-48990-3
Al Mubtasim	22-49002-3
Adiba Tanzila	22-49012-3

TABLE OF CONTENTS

TOPICS	Page no.
Title Page	1
Table of Content	2
1. Introduction	3
2. Case Study	4
3. ER Diagram	5
4. Normalization	6-7
5. Finalization	8
6. Table Creation	9-13
7. Data Insertion	13-15
8. Query Test	16-21
9. Database Connection	22-24
10. Conclusion	25

Introduction:

This is a database project on Doctor Appointment Management System that will store the data of the patients, receptionists, doctors, hospitals, and their branches. This system will help the medical sector to allow them to properly arrange and store their data for different hospitals. The database was initiated in Oracle and the Database connection was done with JAVA and connected it with MySQL. The database is created with 8 tables consisting of different data.

Case Study:

Doctor Appointment Management System

In a Doctor Appointment Management System, a patient can schedule multiple doctors' appointments by calling the receptionist. Patients provide their name, age, gender, and phone number as a unique identifier. The patient pays the receptionist and to confirm payment, trx_id is stored. The Receptionist is identified by a unique ID, Name, and Phone Number. The doctors are distinguished by Name, ID, Phone Number. The system also stores the schedule of each doctor. Doctors can be specialized into cardiologist, pediatrician, neurologist. A doctor has atmost 3 receptionists and a receptionist may be assigned to atmost 2 doctors. Doctors may be affiliated with multiple Hospitals, each with a distinct licence_no, Name, email, and Phone Number. A hospital may have multiple doctors and branches. Branches have ID to be uniquely identified. The system also has the address, name, phone number, email of the Branches.

ER Diagram:

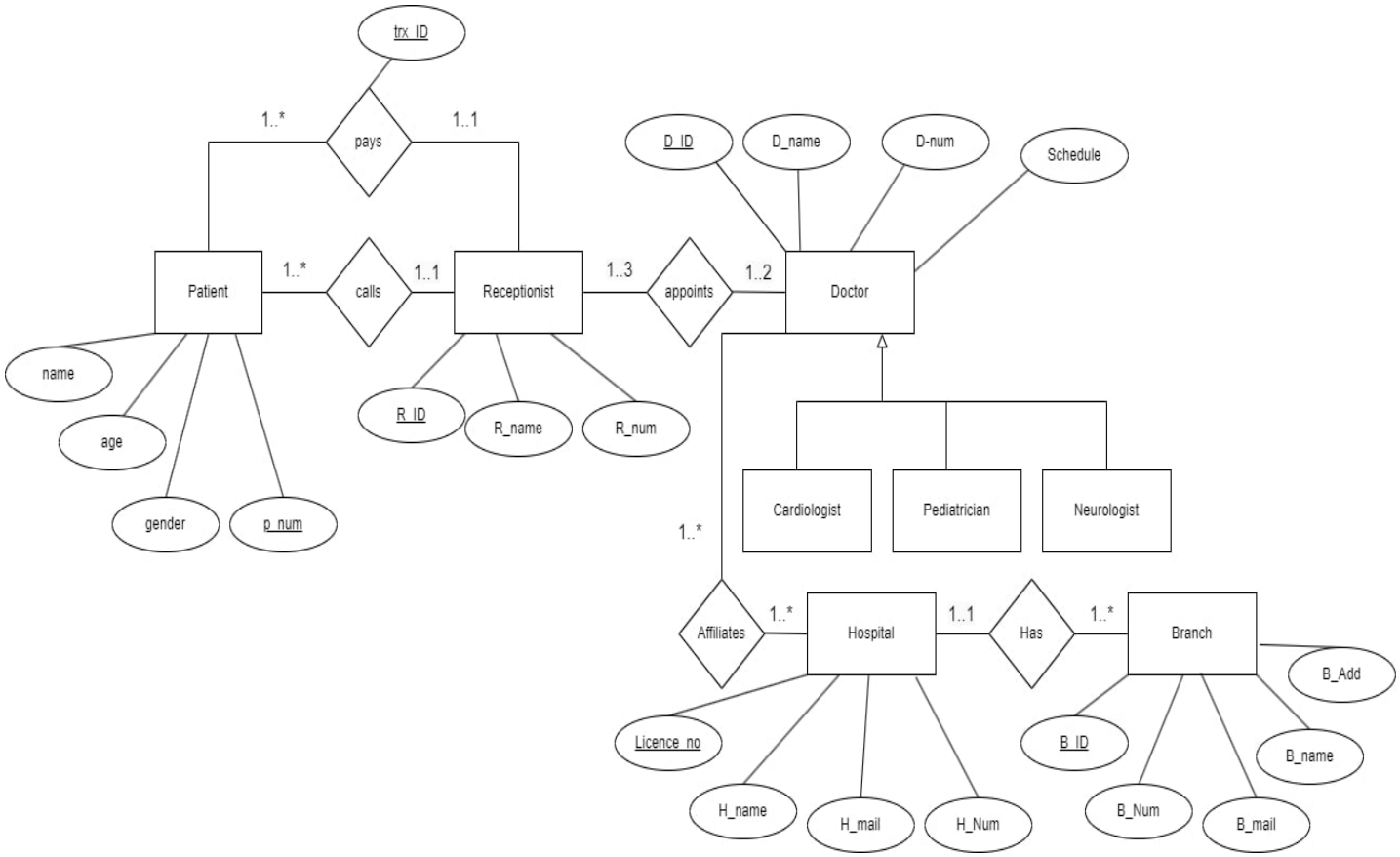

Fig 3: Doctor Appointment Management System

NORMALIZATION:

Call

UNF: name,age,gender,p_num,R_ID,R_name,R_num

1NF: p_num,name,age,gender, R_ID,R_name,R_num

2NF: (1)p_num(PK),gender,age,name,R_ID(FK)

(2)R_id(PK),R_name,R_num

3NF: same as 2NF

Pays

UNF: name,age,gender,p_num,R_ ID,R_name,R_num,trx_ ID

1NF: p_num,name,age,gender, R_ ID,R_name,R_num,trx_ ID

2NF: (1) trx_ ID (PK),p_num(FK),gender,age,name,R_ID(FK)

(2)R_ID(PK),R_name,R_num

3NF: same as 2NF

Appoints

UNF: R_ID,R_name,R_num,D_ID, D_name,D_num,schedule.

1NF: R_ID,R_name,R_num, D_ID, D_name,D_num,schedule

2NF: (1) R_ID(PK),R_name,R_num

(2) D_ID(PK), D_name,D_num,schedule.

(3)R_ID(PK), D_ID(FK)

3NF: same as 2NF

Affiliates

UNF: D_ID, D_name,D_num,schedule,licence_no,H_name,H_mail,H_num

1NF: D_ID, D_name,D_num,schedule,licence_no,H_name,H_mail,H_num

2NF: (1) licence_no(PK),H_name,H_mail,H_num

(2) D_ID(PK), D_name,D_num,schedule.

(3)D_ID(PK),licence_no(FK)

3NF: same as 2NF

Has

UNF: licence_no,H_name,H_mail,H_num,B_ID,B_num,B_name,B_mail,B_add

1NF: licence_no,H_name,H_mail,H_num, B_ID,B_num,B_name,B_mail,B_add

2NF: (1) licence_no(PK),H_name,H_mail,H_num

(2) B_ID,B_num,B_name,B_mail,B_add,licence_no(FK)

3NF: same as 2NF

FINALIZATION

Columns	Tables
(1) p_num(PK),gender,age,name,R_id(FK)	[CALL]
(2) R_id(PK),R_name,R_num	[RECEPTIONIST]
(3) Trx_id(PK), p_num(FK),gender,age,name,R_id(FK)	[PAY]
(4) R_id(PK),R_name,R_num	x
(5) R_id(PK),R_name,R_num	x
(6) D_ID(PK), D_name,D_num,schedule.	[DOCTOR]
(7) R_id(PK), D_ID(FK)	[APPOINT]
(8) licence_no(PK),H_name,H_mail,H_num	[HOSPITAL]
(9) D_ID(PK), D_name,D_num,schedule.	x
(10) D_ID(PK),licence_no(FK)	[AFFILIATE]
(11) licence_no(PK),H_name,H_mail,H_num	x
(12) B_ID(PK),B_num,B_name,B_mail,B_add,licence_no(FK)	[HAS]

Final Tables:

(1) **CALL**: p_num(PK),gender,age,name,R_id(FK)

(2) **RECEPTIONIST**: R_id(PK),R_name,R_num

(3) **PAY:** Trx_id(PK), p_num(FK),gender,age,name,R_id(FK)

(4) **DOCTOR:** D_ID(PK), D_name,D_num,schedule.

(5) **APPOINT:** R_id(PK), D_ID(FK)

(6) **HOSPITAL:** licence_no(PK),H_name,H_mail,H_num

(7) **AFFILIATE:** D_ID(PK),licence_no(FK)

(8) **HAS:** B_ID(PK),B_num,B_name,B_mail,B_add,licence_no(FK)

Table Creation:

Call table:

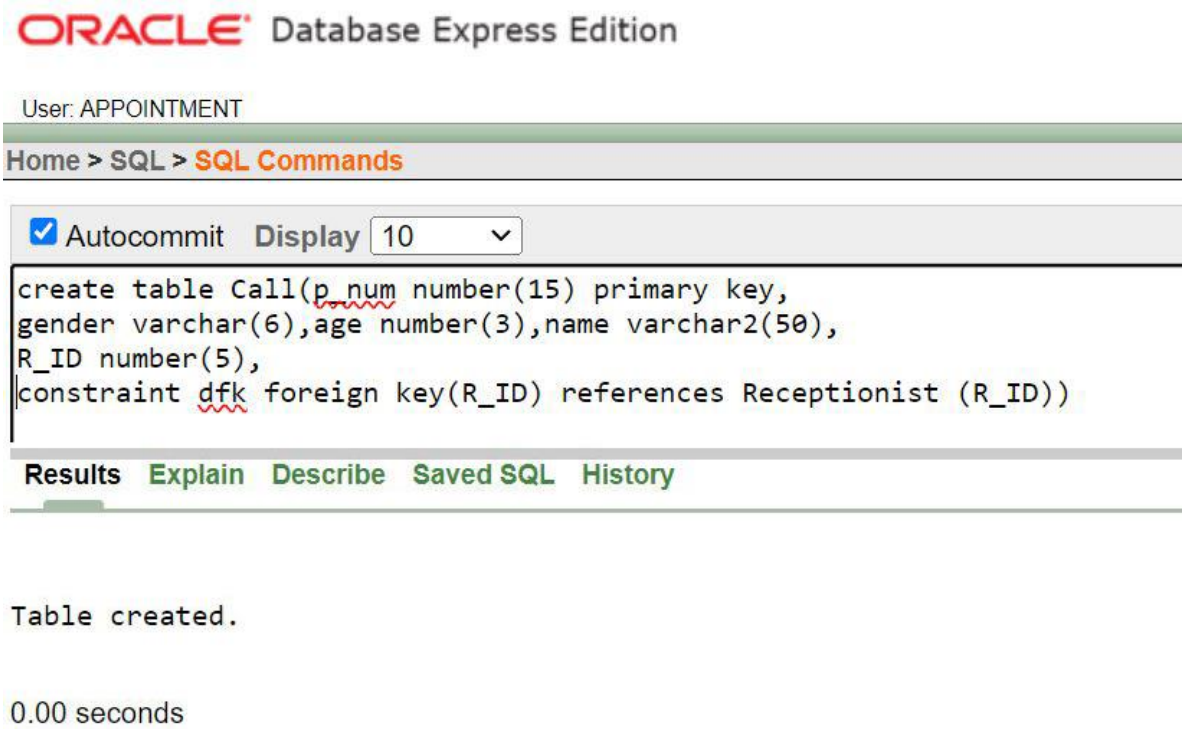

Fig 6.1: Call table creation

Receptionist table:

ORACLE Database Express Edition

User: APPOINTMENT

Home > SQL > SQL Commands

Autocommit Display 10

```
create table Receptionist(R_ID number(5) primary key,
R_name varchar2(50),R_num number(15))
```

Results Explain Describe Saved SQL History

```
Table created.
```

0.00 seconds

Fig 6.2: Receptionist table creation

Pay Table:

ORACLE Database Express Edition

User: APPOINTMENT

Home > SQL > SQL Commands

Autocommit Display 10

```
create table Pay (trx_ID number(5) primary key,gender varchar(6), age number(3),
name varchar2(50),R_ID number(5),
constraint efk foreign key(R_ID) references Receptionist (R_ID),
p_num  number(15),constraint ffk foreign key(p_num ) references Call(p_num ))
```

Results Explain Describe Saved SQL History

```
Table created.
```

0.02 seconds

Fig 6.3: Pay table creation

Doctor Table:

ORACLE Database Express Edition

User: APPOINTMENT

Home > SQL > SQL Commands

Autocommit Display 10

```
create table doctor( D_ID number(5) primary key,D_name varchar2(50),D_num number(15),schedule varchar2(15))
```

Results Explain Describe Saved SQL History

```
Table created.
```

0.03 seconds

Fig 6.4: Doctor table creation

Appoint Table:

ORACLE Database Express Edition

User: APPOINTMENT

Home > SQL > SQL Commands

Autocommit Display 10

```
create table appoint( R_ID number (5)primary key, D_ID number(5),constraint afk foreign key(D_ID) references doctor (D_ID))
```

Results Explain Describe Saved SQL History

```
Table created.
```

0.00 seconds

Fig 6.5: Appoint table creation

Hospital Table:

ORACLE Database Express Edition

User: APPOINTMENT

Home > SQL > SQL Commands

Autocommit Display 10

```
create table hospital (licence_no number(8) primary key,H_name varchar2(50), H_mail varchar(40), H_num number(15))
```

Results Explain Describe Saved SQL History

```
Table created.
```

0.00 seconds

Fig 6.6: Hospital Table Creation

Affiliate Table:

ORACLE Database Express Edition

User: APPOINTMENT

Home > SQL > SQL Commands

Autocommit Display 10

```
create table affiliate (D_ID number(5) primary key,licence_no number(8),
constraint cfk foreign key(licence_no) references hospital (licence_no))
```

Results Explain Describe Saved SQL History

```
Table created.
```

0.00 seconds

Fig 6.7: Affiliate Table Creation

Has Table:

ORACLE Database Express Edition

User: APPOINTMENT

Home > SQL > SQL Commands

Autocommit Display 10

```
create table has(B_ID number(5) primary key,B_num number (15),
 B_name varchar2(50),B_mail varchar(40), B_add varchar2(100),
licence_no number(8), constraint bfk foreign key(licence_no)
references hospital(licence_no))
```

Results Explain Describe Saved SQL History

```
Table created.
```

0.00 seconds

Fig 6.8: Has Table Creation

Value Insert:

Affiliate Table:

D_ID	LICENCE_NO
104	86067
103	75056
102	64045
101	53034

4 rows returned in 0.00 seconds CSV Export

Fig 7.1: Value insertion in Affiliate Table

Appoint Table:

R_ID	D_ID
201	101
202	102
203	103
204	104

4 rows returned in 0.00 seconds CSV Export

Fig 7.2: Value insertion in Appoint Table

Call Table:

P_NUM	GENDER	AGE	NAME	R_ID
8801746897245	MALE	25	Mahfuz Hasan	201
8801826885114	FEMALE	37	Ayesha Khatun	202
8801526780157	MALE	42	Zunayed Islam	203
8801451491215	FEMALE	29	Sabina Islam	204

4 rows returned in 0.00 seconds CSV Export

Fig 7.3: Value insertion in Call Table

Doctor Table:

D_ID	D_NAME	D_NUM	SCHEDULE
102	Dr. Tofazzal Hossain	8801834257284	SUN-TUE
101	Dr. Zahidul Islam	8801913672845	SAT-THU
103	Dr. Munira Ferdousi	8801553859235	MON-THU
104	Dr. Umme Salma	8801957249107	MON-THU

4 rows returned in 0.00 seconds CSV Export

Fig 7.4 :Value insertion in Doctor Table

Has Table:

B_ID	B_NUM	B_NAME	B_MAIL	B_ADD	LICENCE_NO
43035	96137878051	Popular Diagnostics Center Ltd-UTTARA	popular_udc@gmail.com	Uttara	53034
54046	96100096121	Ibn Sina Diagnostic Center-Banani	ibn_sinaban@gmail.com	BANANI	64045
65057	96454047291	Uttara Cresent Hospital-2	uc2_resent@gmail.com	Sector-7	75056
76068	92070738591	Labaid Diagnostics Uttara-2	lab2_aid@gmail.com	Sector-13	86067

4 rows returned in 0.00 seconds CSV Export

Fig 7.5: Value insertion in Has Table

Hospital Table:

LICENCE_NO	H_NAME	H_MAIL	H_NUM
53034	Popular Diagnostics Center Ltd	popular_dc@gmail.com	9613787805
64045	Ibn Sina Diagnostic Center	ibn_sina@gmail.com	9610009612
75056	Uttara Cresent Hospital	uc_resent@gmail.com	9645404729
86067	Labaid Diagnostics Uttara	labaid_diag@gmail.com	9207073859

4 rows returned in 0.00 seconds CSV Export

Fig 7.6: Value insertion in Hospital Table

Pay Table:

TRX_ID	GENDER	AGE	NAME	R_ID	P_NUM
56129	MALE	25	Mahfuz Hasan	201	8801746897245
78340	FEMALE	37	Ayesha Khatun	202	8801826885114
89563	MALE	42	Zunayed Islam	203	8801526780157
10781	FEMALE	29	Sabina Islam	204	8801451491215

4 rows returned in 0.00 seconds CSV Export

Fig 7.7: Value insertion in Pay Table

Receptionist Table:

R_ID	R_NAME	R_NUM
201	Mahfuzul Islam	8801437835923
204	Ahmed Karim	8801642457431
203	Farhan Ahmed	8801872982285
202	Zihad Haque	8801985252336

4 rows returned in 0.00 seconds CSV Export

Fig 7.8: Value insertion in Receptionist Table

Query Test:

(1)Simple Query:

Show the age of patient named 'Ayesha Khatun'

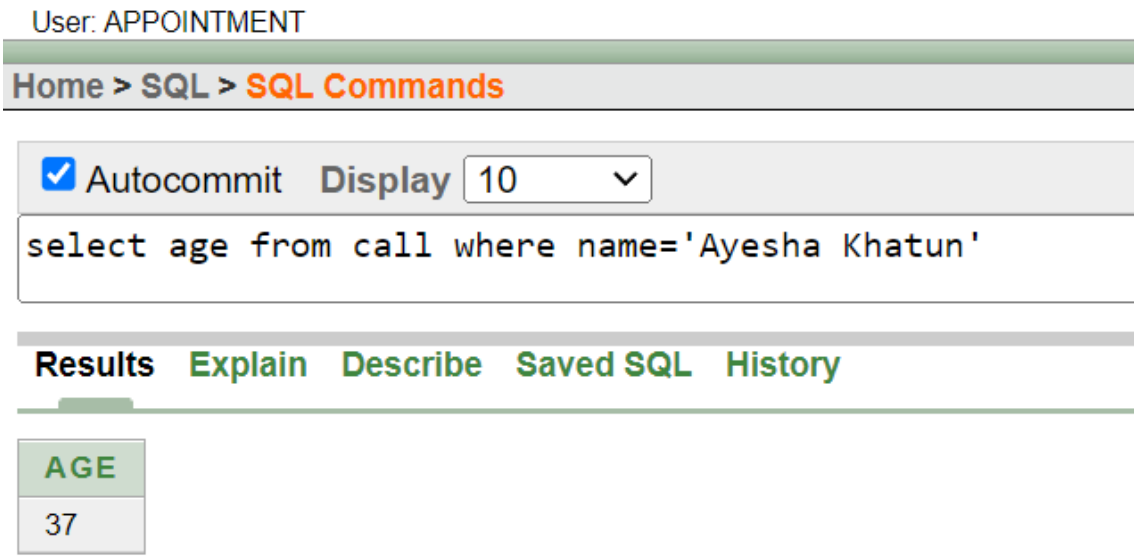

Fig 8.1: Query Execution: Simple Query

(2)Single Row Function Query:

Find the b_name,b_add and length of the b_name which is in Uttara.

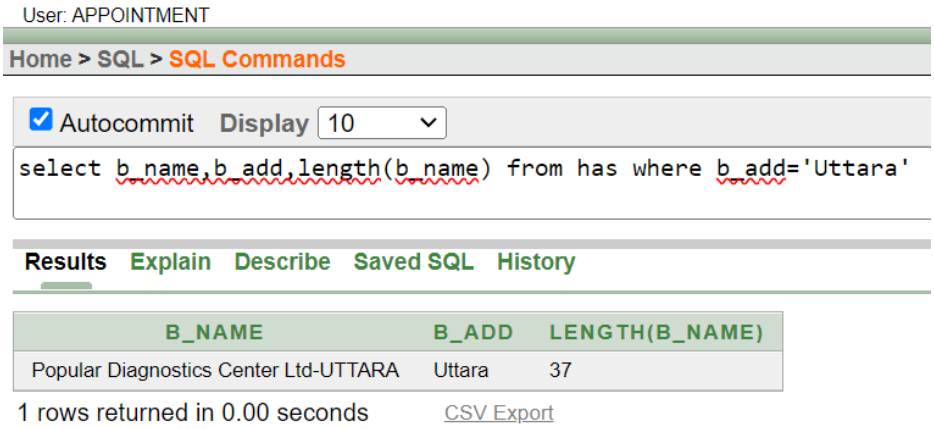

Fig 8.2: Query Execution: Single Row Function Query

(3)Group Function Query:

Find the age of the oldest Patient.

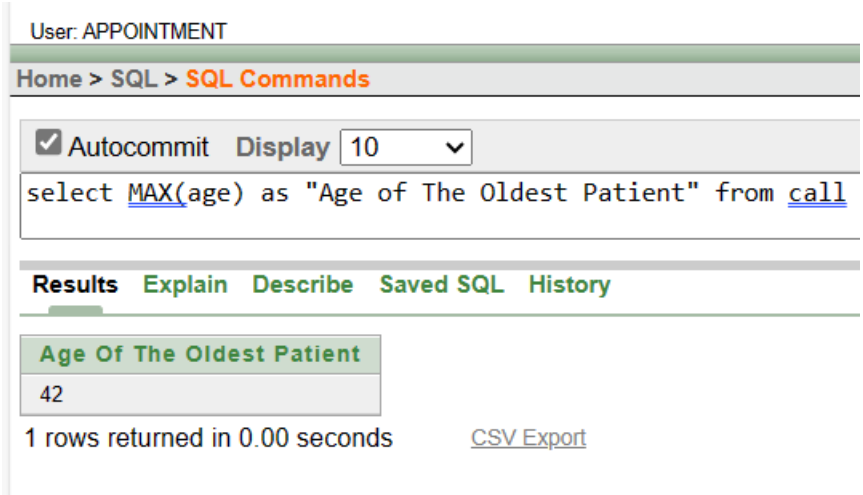

Fig 8.3: Query Execution: Group Function Query

(4)Single Row Subquery:

Show the name of the patient who is younger than Sabina Islam

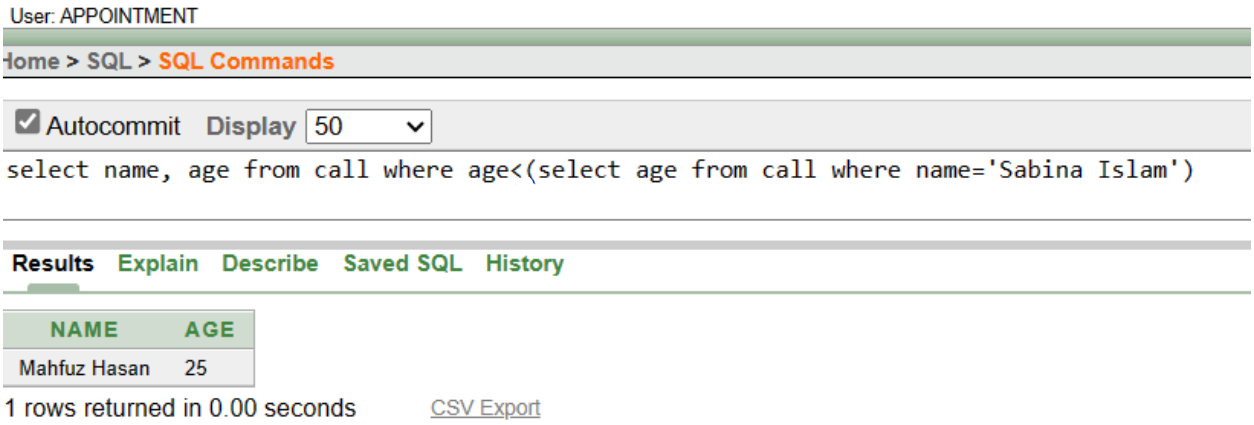

Fig 8.4: Query Execution: Single Row Subquery

(5) Multiple Row Subquery:

Show the name of the doctors who do not have the same Schedule as Dr. Munira Ferdousi

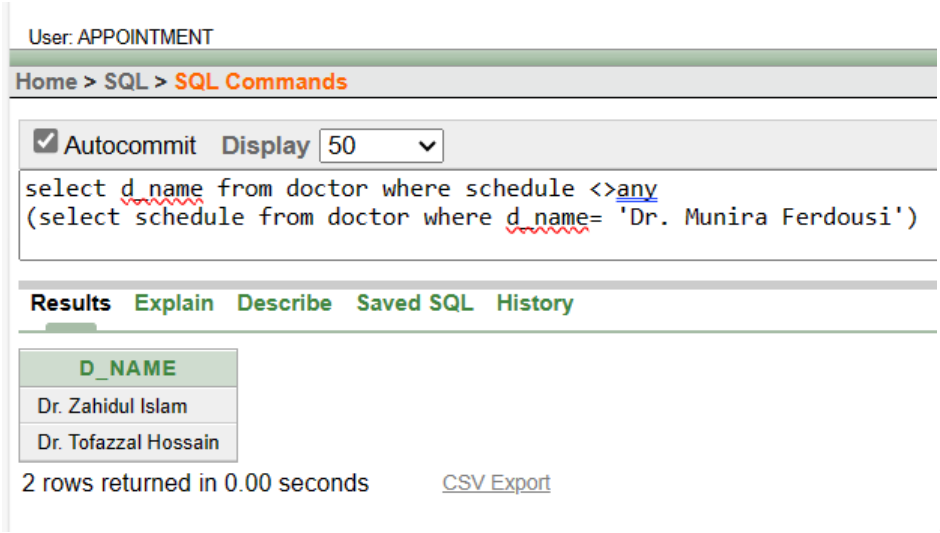

Fig 8.5: Query Execution: Multiple Row Subquery

(6.1)Joining- Equijoin:

Join appoint and receptionist table where r_id is equal.

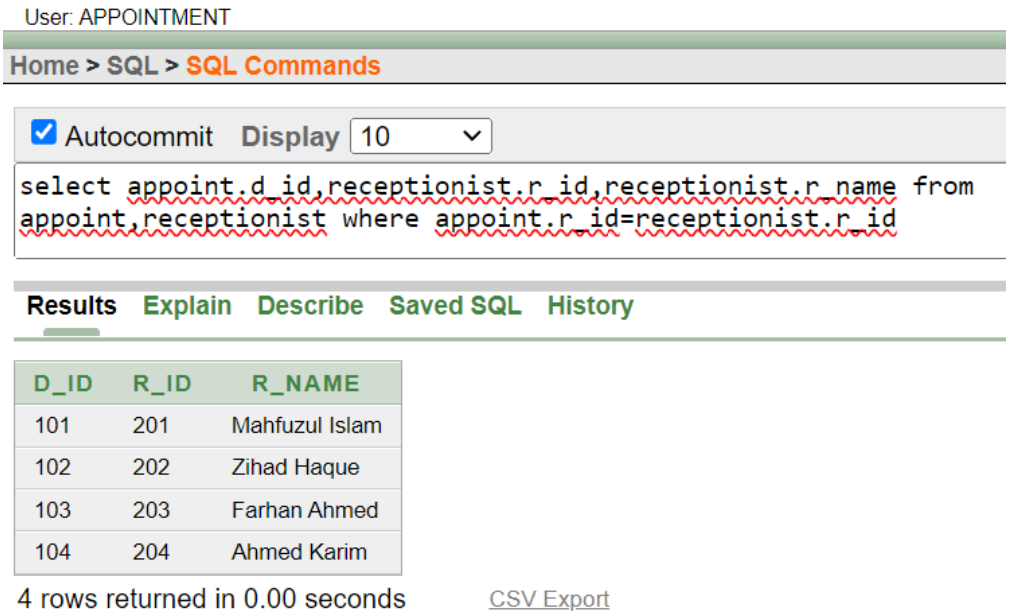

Fig 8.6.1: Query Execution: Equijoin

(6.2) Joining- Self Join:

Show Payment Detail with the name of the patient and trx_id using selfjoin.

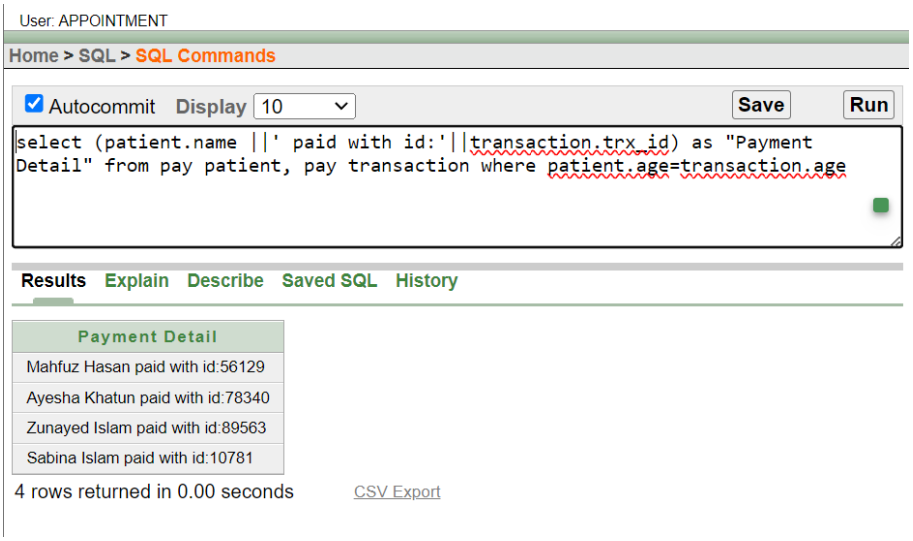

Fig 8.6.2: Query Execution: Self-Join

(7.1)Simple View:

Create a view named Malepatient where the gender of the patient will be MALE.

Query: create view Malepatient as select name,gender from call where gender='MALE'

User: APPOINTMENT

Home > SQL > SQL Commands

Autocommit Display 10

```
create view Malepatient as select name,gender from call where gender='MALE'
```

Fig 8.7.1.1: Simple view creation command

Object Type **VIEW** Object **MALEPATIENT**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
MALEPATIENT	NAME	Varchar2	50	-	-	-	✓	-	-
	GENDER	Varchar2	6	-	-	-	✓	-	-

1 - 2

Fig 8.7.1.2: Description of the simple view

Results Explain Describe Saved SQL History

NAME	GENDER
Mahfuz Hasan	MALE
Zunayed Islam	MALE

2 rows returned in 0.00 seconds CSV Export

Fig 8.7.1.3:Result of the simple view as a whole table

(7.2)Complex View:

Create a view of the Pay table which shows name and trx_id with Self-join

Query: create view TransactionInfo("Payment Detail") as select patient.name ||' paid with id:'||transaction.trx_id from pay patient, pay transaction where patient.age=transaction.age

User: APPOINTMENT

Home > SQL > SQL Commands

Autocommit Display 10

```
create view TransactionInfo("Payment Detail") as
select patient.name ||' paid with id:'||transaction.trx_id
from pay patient, pay transaction where patient.age=transaction.age
```

Fig 8.7.2.1: Complex view creation command

Results Explain Describe Saved SQL History

Object Type **VIEW** Object **TRANSACTIONINFO**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
TRANSACTIONINFO	Payment Detail	Varchar2	104	-	-	-	✓	-	-
									1 - 1

Fig 8.7.2.2: Description of the complex view

Results Explain Describe Saved SQL History

Payment Detail
Mahfuz Hasan paid with id:56129
Ayesha Khatun paid with id:78340
Zunayed Islam paid with id:89563
Sabina Islam paid with id:10781

4 rows returned in 0.00 seconds CSV Export

Fig 8.7.2.3:Result of the complex view as a whole table

Database Connection:

(Al Mubtasim)

IDE used: Apache Netbeans 20

Database Management System: MySQL

Java Version: 21.0.1

MySQL Connector/J: JDBC jar file version: 8.0.28

Webserver: Apache Xampp version 8.2.12

Procedure:

1. First download and install the mentioned softwares.
2. Then, the JAR file was kept in any drive's (except C:drive) new folder.
3. Open Xampp and start Apache and MySQL together and then click on admin beside MySQL
4. Create a Database and then create a table with 4 columns.
5. Add name,datatype and size of the columns and then insert the values into the table.
6. Open Netbeans IDE and create a new project,then add the jar file in the libraries by locating the jar file.
7. Write the Database connection code and print output.
8. The connection between the Database and the Java code has been established.

D_ID	D_NAME	D_num	SCHEDULE
101	Dr. Zahidul Islam	01913672845	SAT-THU
102	Dr. Tofazzal Hossain	01834257284	SUN-TUE
103	Dr. Munira Ferdousi	01553859235	MON-THU
104	Dr. Umme Salma	01957249107	MON-THU

Fig 9.1:Query Execution of Doctor table in MySQL

```
Connection Established
D_ID: 101, D_NAME: Dr. Zahidul Islam, D_num: 01913672845, SCHEDULE: SAT-THU
D_ID: 102, D_NAME: Dr. Tofazzal Hossain, D_num: 01834257284, SCHEDULE: SUN-TUE
D_ID: 103, D_NAME: Dr. Munira Ferdousi, D_num: 01553859235, SCHEDULE: MON-THU
D_ID: 104, D_NAME: Dr. Umme Salma, D_num: 01957249107, SCHEDULE: MON-THU
BUILD SUCCESSFUL (total time: 0 seconds)
```

Fig 9.2:Output in Netbeans IDE : Database connection

(Shahriar Hossain)

Java version: JDK 21

Connector:JDBC jar file 8.0.28

IDE: Apache NEtbeans 18

WebServer:Apache 2.4.56

Procedure:

1.After installing all the mentioned software we have to open Xampp and where we have to start Apache and MySQL at a time and go to admin of MySQL.

2.where we have to create Database and table and then insert values.

3.Then in Netbeans IDE we have to create a project where we have to connect the jar file we downloaded using the libraries option.

4.Then we have to do the database connection code and run it.

H_name	H_number	H_mail	licience_no
Popular Diagnostics Center Ltd	9613787805	popular_dc@gmail.com	53034
Ibn Sina Diagnostic Center	96100009612	ibn_sina@gmail.com	64045
Uttara Cresent Hospital	96218959283	uc_resent@gmail.com	75056
Labaid Diagnostics Uttara	96382100034	labaid_diag@gmail.com	86067

Fig 9.3: Hospital table in MySQL

```
Output - Apppointment (run)
run:
H_namePopular Diagnostics Center Ltd H_number9613787805 H_mailpopular_dc@gmail.com licience_no53034
H_nameIbn Sina Diagnostic Center H_number96100009612 H_mailibn_sina@gmail.com licience_no64045
H_nameUttara Cresent Hospital H_number96218959283 H_mailuc_resent@gmail.com licience_no75056
H_nameLabaid Diagnostics Uttara H_number96382100034 H_maillabaid_diag@gmail.com licience_no86067
Connected
BUILD SUCCESSFUL (total time: 0 seconds)
```

Fig 9.4:Output from Apache

(Adiba Tanzila)

IDE: Apache NetBeans 20

Dtabased Management System: MySQL

Java Version: 19.0.2

MySQL Connector: JDBC Jar 8.0.28

Webserver: Apache Xampp 8.2.12

Procedure:

1. Download all the mentioned apps.
2. Enter the C drive and open Xampp file.
3. Open start MySQL and Apache start button also press MySQL admin.
4. Create Patient then create a table with 5 columns and Insert all the values from table
5. Open NetBeans create a project then run the Database connecting code with java.
6. Then take a snap from code output.

P_NUM	GENDER	AGE	NAME	R_ID
01451491215	FEMALE	29	Sabina Islam	204
01526780157	MALE	42	zunayed Islam	203
01746897245	MALE	25	Mahfuz Hasan	201
01826885114	FEMALE	37	Ayesha Khatun	202

Fig 9.5: Call table in MySQL

Call Table:

```
Connected
P_NUM=01746897245 GENDER=MALE AGE=25 NAME=Mahfuz Hasan  R_ID=201
P_NUM=01826885114 GENDER=FEMALE AGE=37 NAME=Ayesha Khatun R_ID=202
P_NUM=01526780157 GENDER=MALE AGE=42 NAME=zunayed Islam R_ID=203
P_NUM=01451491215 GENDER=FEMALE AGE=29 NAME=Sabina Islam R_ID=204
BUILD SUCCESSFUL (total time: 0 seconds)
```

Fig 9.6: Java output

Conclusion:

The “Doctor Appointment Management System” can store various data about different Hospitals, Branches, Doctors, Receptionists and Patients and also constantly update data and information about them. The data can be properly organized in different tables and easily accessed. This can also be used throught Java applications.